Assignment 1 Report: Frequent Mining Experiments
Q1 (Frequent Itemsets): Apriori vs. FP-Growth

COL761/COL7361/AIL7026 Data Mining

February 2, 2026

1 Introduction

Frequent itemset mining is a fundamental problem in data mining that discovers patterns of items frequently occurring together in transaction databases. This is a crucial step in association rule learning and market basket analysis. Two prominent algorithms for this task are the Apriori algorithm and the FP-Growth (Frequent Pattern Growth) algorithm.

This report presents an empirical comparison of these two algorithms on the WebDocs dataset, followed by an analysis of a synthetic dataset designed to exhibit specific runtime characteristics.

All experiments were run using the provided Bash pipelines in the repository (Task 1: `q1_1.sh`, Task 2: `q1_2.sh`) and timed using wall-clock time around the executable invocations. The timing therefore includes dataset parsing/reading, internal preprocessing performed by the tools, and the frequent itemset mining + output writing.

2 Task 1: Algorithm Comparison on WebDocs Dataset

2.1 Dataset Characteristics

The WebDocs dataset was obtained from the FIMI (Frequent Itemset Mining Implementations) repository. This real-world dataset represents web page collections with the following characteristics:

- **Total items:** 5,267,656 (item occurrences)

- **Transactions:** 1,692,082 (web documents)
- **Unique items after filtering:** 262 items
- **Reduced transactions:** 1,319,600 transactions (after redundancy removal)

The dataset is dense and exhibits complex item correlations typical of real-world web document data.

2.2 Experimental Setup

Both algorithms were intended to be executed at five different support thresholds: 5%, 10%, 25%, 50%, and 90%. In the collected run artifacts for WebDocs (stored under `A1/q1/output_q1_1/`), we have results for 10%, 25%, 50% and 90%; the 5% WebDocs run is not present (the corresponding output/log files are absent). Since WebDocs is very large, 5% is typically the most expensive regime and is commonly run on a more powerful machine (as also suggested in the assignment tips).

In our attempt to run WebDocs at 5% support, the pipeline hit a timeout (i.e., exceeded the allowed runtime budget) and therefore did not produce complete tool logs or outputs for that threshold. Consequently, we explicitly mark the 5% datapoint as unavailable and focus the detailed analysis on the completed thresholds.

The runtime measurements reported below include all preprocessing steps (reading, filtering, item recoding, transaction reduction, and tree construction where applicable) as well as the core itemset mining computation and output writing.

2.3 Results Analysis

2.3.1 Quantitative Results

Support (%)	Apriori (s)	FP-Growth (s)	Speedup
5	—	—	—
10	355.67	65.21	5.45×
25	21.28	21.74	0.98×
50	20.52	20.12	1.02×
90	19.96	18.21	1.10×

Table 1: Runtime Comparison on WebDocs Dataset

Note: The WebDocs 5% datapoint is not available in the current run logs; the rest of the report focuses on the available thresholds.

2.3.2 Log-Derived Mining Statistics (WebDocs)

Besides wall-clock runtimes, the tool logs provide additional signals that help explain performance: how many items survive filtering, how many transactions remain after reduction, how large the internal transaction tree is (Apriori implementation), the maximum size of itemsets explored, and the total number of frequent itemsets output.

Support (%)	Status	Items After Filter	Reduced Txns	Tree Nodes (Apriori)	#Freq. Sets
5	timeout (no logs)	—	—	—	—
10	completed	262	1,319,600 / 1,692,082	2,317,962	217,767
25	completed	29	742,260 / 1,692,082	1,303,240	413
50	completed	5	32 / 1,692,082	32	10
90	completed (early exit)	0 (no frequent items)	—	—	0

Table 2: WebDocs: statistics extracted from tool logs (filtering, reduction, and output counts).

2.3.3 Key Observations

1. **Dramatic speedup in the low-support regime:** At 10% support, FP-Growth is approximately 5.45 times faster than Apriori (65.21s vs 355.67s). This is the most significant performance gap observed in the available datapoints.

2. **Convergence at high support:** As the support threshold increases to 25%, 50%, and 90%, the performance difference diminishes. At these supports the number of frequent items/patterns collapses sharply, so both algorithms spend most time just reading and preprocessing the dataset.

3. **Itemset counts collapse with support:** At 10% support, 217,767 frequent itemsets are discovered and Apriori explores itemsets up to size 11. At 25%, only 413 frequent itemsets remain. At 50%, only 10 frequent itemsets remain. At 90%, the tool reports *no frequent items found* (i.e., even 1-itemsets fail the threshold).

2.3.4 Runtime Decomposition from Tool Logs

The tool-generated logs provide a helpful breakdown. Two takeaways stand out: (i) at 10% support, Apriori spends the majority of its time in sub-

set checking (candidate counting over multiple levels), whereas FP-Growth spends most of its post-read time in mining/writing; and (ii) at 25% and above, runtimes are dominated by dataset reading/parsing and internal preprocessing, so both algorithms look similar.

For example (WebDocs, 10%):

- **Apriori:** reading 17.84s; filtering 0.92s; reduction 2.21s; tree build 0.36s; subset checking 334.00s; output write 0.02s.
- **FP-Growth:** reading 18.55s; filtering 1.10s; reduction 2.82s; output write (includes mining) 42.52s.

More detailed per-threshold interpretation (WebDocs).

- **10% ("hard" regime):** 262 items survive filtering and 217,767 frequent itemsets are output. Apriori explores itemsets up to size 11 and spends 334s in subset checking, which is *well over 90%* of its log-reported runtime. FP-Growth, in contrast, spends the bulk of its time in the final "writing" phase (42.52s), which in Borgelt's implementation effectively includes the recursive mining plus output emission. The key point is that the mining workload is large (many patterns), and Apriori pays it as repeated candidate counting across levels.
- **25% (transition):** only 29 items survive filtering and only 413 frequent itemsets remain; Apriori now explores only up to size 6. The expensive part collapses: subset checking drops to 1.74s. Both algorithms become dominated by reading/parsing and preprocessing the large WebDocs file (`reading` alone is ∼18–19s).
- **50% (degenerate after reduction):** only 5 items survive filtering, and reduction collapses the dataset to just 32 transactions. Subset checking becomes essentially zero, and both algorithms look similar overall because they still must read the original dataset from disk to determine item supports.
- **90% (early termination):** both tools report *no frequent items found* right after reading the dataset. This highlights an often-missed fact: even when the mining result is empty, there is still a fixed cost to stream and count item occurrences in the input. Hence the runtime remains ∼18–20s despite producing no itemsets.

Why the "transaction reduction" lines matter. Both tools sort and reduce transactions after filtering/recoding items. This step removes redundant/duplicate transactions and can dramatically shrink the working

database (e.g., WebDocs reduces from 1,692,082 to 1,319,600 transactions at 10%). Lower supports keep more items, which increases transaction lengths but also increases opportunities for redundancy removal and compression in the internal representation (seen indirectly via the transaction-tree node counts in Apriori logs).

At very high supports, the reduction step can also collapse the dataset because most transactions lose all items after filtering and are dropped. This is especially visible at 50% support on WebDocs: only 5 items survive filtering and only 32 transactions remain after reduction, so mining becomes trivial even though the input file still must be scanned.

Why 5% is the problematic WebDocs point. Moving from 10% to 5% would (typically) allow many more items to pass filtering, increase the maximum frequent itemset size explored, and drastically increase the number of candidate levels (and the work in Apriori's subset-checking phase). This aligns with the observed timeout in our attempted 5% run.

2.4 Algorithm Characteristics and Explanation

2.4.1 Apriori Algorithm

The Apriori algorithm generates frequent itemsets level-by-level using a bottom-up approach:

1. Discovers frequent 1-itemsets by scanning the database
2. Iteratively generates candidates of size $(k + 1)$ from frequent itemsets of size k
3. Uses the anti-monotone property: if an itemset is infrequent, all its supersets must be infrequent
4. Prunes candidates that fail support threshold via database scans

Complexity factors:

- At low support thresholds, the candidate space explodes exponentially
- For the WebDocs dataset at 10% support, over 217,000 frequent itemsets exist
- Each level requires a complete database scan for candidate support counting
- With many frequent items, combinatorial candidate generation becomes prohibitively expensive

2.4.2 FP-Growth Algorithm

FP-Growth employs a pattern growth approach using a compact tree structure:

1. Builds an FP-tree (Frequent Pattern tree) that compresses the transaction database
2. Recursively mines patterns from the tree without candidate generation
3. Uses a novel divide-and-conquer strategy by mining conditional pattern bases
4. Avoids expensive candidate generation and multiple database scans

Efficiency advantages:

- Database is scanned only twice (once for preprocessing, once for tree construction)
- No explicit candidate generation or pruning
- FP-tree provides compact representation, reducing memory overhead
- Tree structure exploitation enables efficient pattern growth

2.5 Performance Analysis

The dramatic speedup at low support thresholds reflects the algorithmic differences:

- **Candidate explosion (Apriori) vs. pattern growth (FP-Growth):** At 10% support, 262 items survive filtering and 217,767 frequent itemsets exist. Apriori's level-wise candidate counting is the dominant cost (334s in subset checking), while FP-Growth avoids explicit candidate generation.
- **Why runtimes become similar at higher supports:** As support rises, the number of surviving items shrinks drastically (29 items at 25%, 5 items at 50%, and none at 90%). In this regime, the mining phase is trivial for both methods, so the measured wall-clock time is largely the cost to read and preprocess the large input file.
- **Output-size effects still matter:** Even for FP-Growth, enumerating and writing large numbers of itemsets is a real cost (e.g., 42.52s in the 10% run is reported in the final "writing" step).

2.6 Runtime Plots (WebDocs)

Figure 1 shows the runtime curve produced by the pipeline for the available thresholds.

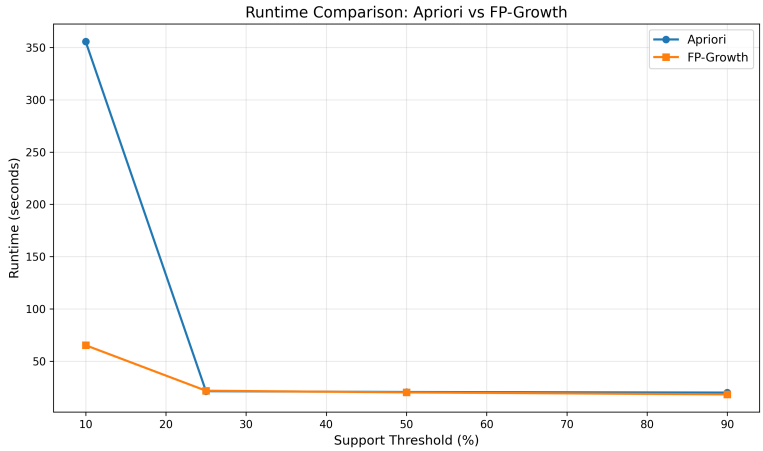

Figure 1: Runtime comparison on WebDocs (available thresholds from `output_q1_1/results.txt`). X-axis: support threshold (%). Y-axis: runtime (seconds).

3 Task 2: Synthetic Dataset Analysis

3.1 Dataset Generation Strategy

For Task 2, we generated a synthetic dataset with 15,000 transactions from a universal itemset of 26 items. The generator (`A1/q1/generate_transactions.py`) is designed to create a dense, highly-correlated "core" of items plus lower-frequency noise, so that Apriori experiences heavy candidate/subset-checking cost while FP-Growth remains comparatively efficient.

3.1.1 Generative Model

The dataset generation employs the following strategy to create correlated item patterns (specializing to the actual `universal_itemset=26` run):

1. **Correlated clusters (core items):** With 26 items, the generator forms *two* correlated clusters of size 10 each (20 "core" items total), plus 6 remaining "noise" items.

2. **Cluster supports:** Clusters are injected with high probability (80% and 70% in our 2-cluster case), creating transactions that frequently contain many (often all) of the core items together.

3. **Inter-cluster correlations:** With 70% probability, the generator additionally samples items from multiple clusters in the same transaction, amplifying cross-cluster co-occurrence and creating many frequent combinations.

4. **Partial membership:** When a cluster is not fully added, items are still included independently with 50% probability, preventing the dataset from becoming perfectly deterministic while keeping supports high.

5. **Noise items:** A few remaining items are injected at low probability (6%) to create a small number of additional frequent patterns at very low supports.

This structure ensures that at moderate support levels (5-50%), many correlated itemsets exist, creating a high candidate load for Apriori while FP-Growth's tree structure handles this efficiently.

3.1.2 Dataset Characteristics

- **Transactions:** 15,000
- **Universal itemset:** 26 items
- **Transaction density:** Varies based on cluster membership and inter-cluster correlations
- **Correlation structure:** Strong intra-cluster correlations, moderate inter-cluster correlations

3.2 Experimental Results on Synthetic Dataset

3.2.1 Quantitative Results

A striking property of this synthetic dataset is that for supports 10%/25%/50%, exactly 20 items survive filtering and *all* their non-empty subsets are frequent, giving $2^{20} - 1 = 1{,}048{,}575$ frequent itemsets. This creates a very large frequent-pattern output while still keeping the dataset small enough (15,000 transactions) to run quickly.

Support (%)	Apriori (s)	FP-Growth (s)	Speedup	#Freq. Sets
5	2.39	0.39	6.13×	1,049,957
10	2.20	0.28	7.86×	1,048,575
25	2.21	0.28	7.89×	1,048,575
50	2.23	0.28	7.96×	1,048,575
90	0.03	0.03	0.98×	18

Table 3: Runtime Comparison on Synthetic Dataset

3.2.2 Log-Derived Statistics (Synthetic)

The logs for the synthetic runs show not only the final itemset counts, but also how aggressively the tools reduce duplicate transactions and how far Apriori must iterate over itemset sizes:

- **5%:** all 26 items survive filtering; transaction reduction shrinks 15,000 transactions down to 3,620; the Apriori transaction tree has 6,838 nodes; Apriori checks subset sizes up to 20 and outputs 1,049,957 frequent itemsets.
- **10%/25%/50%:** exactly 20 items survive filtering; reduction shrinks to 2,368 transactions; the Apriori transaction tree has 4,242 nodes; Apriori checks subset sizes up to 20 and outputs exactly $2^{20} - 1 =$ 1,048,575 itemsets.
- **90%:** 18 items survive filtering; transaction reduction shrinks to 1,756 transactions; the Apriori transaction tree has 3,022 nodes; only 18 frequent itemsets are output (singletons). The Apriori log shows subset checking only for sizes 1 and 2, consistent with there being no frequent pairs at this extreme support.

Why the "all-subsets" effect happens. If a transaction database contains a large set C of items that co-occur together in at least θ fraction of transactions, then for any non-empty subset $S \subseteq C$ we automatically have

$$\text{supp}(S) \geq \text{supp}(C) \geq \theta,$$

so *every* subset becomes frequent. In our runs, the generator creates a 20-item correlated core that co-occurs frequently enough to clear supports from 10% up through 50%, which forces the output to contain all $2^{20} - 1$ non-empty subsets.

Interpreting 5% vs. 10%+ counts. At 5% support, additional "noise" items (beyond the 20 core items) become frequent and contribute extra frequent itemsets beyond $2^{20} - 1$, which is why the 5% count (1,049,957) is slightly larger than 1,048,575.

3.3 Comparative Analysis: WebDocs vs. Synthetic Dataset

3.3.1 Observed Trends

1. **Consistent FP-Growth advantage:** FP-Growth maintains a significant speedup (6-8×) across support levels 5-50% on the synthetic dataset, compared to 5.45× at 10% on WebDocs

2. **Different speedup profiles:**

   - **WebDocs:** Speedup decreases sharply as support increases (5.45× → 1.10×)
   - **Synthetic:** Speedup remains relatively stable (6-8×) across 5-50% support, then drops dramatically at 90%

3. **Runtime plateau:** On the synthetic dataset, Apriori runtime remains roughly constant (∼2.2s) across 5–50% support thresholds because the set of frequent items (and hence the frequent-itemset search space) changes little across these thresholds.

4. **Sparse regime:** At 90% support on synthetic data, both algorithms execute extremely quickly (∼0.03s) with minimal speedup

3.4 Explanation Based on Item Distribution

3.4.1 WebDocs Dataset Characteristics

- **Item frequency distribution:** The dataset has a natural Zipfian distribution with highly frequent items that become dominant at higher support thresholds

- **Correlation complexity:** Item correlations in WebDocs are diverse and complex, varying significantly across the itemset space

- **Support threshold effect:** As support increases, the fraction of frequent items decreases, reducing both Apriori's candidate generation burden and the FP-tree depth, leading to convergence in runtime

3.4.2 Synthetic Dataset Characteristics

- **Uniform cluster structure:** Items are organized into clusters with controlled, uniform correlation patterns
- **Predictable supports and "all-subsets" effect:** For the chosen generator parameters, a core group of items remains frequent across 10–50% thresholds, and their combinations are frequent as well. This creates a dense frequent-itemset lattice (over one million frequent itemsets).
- **Apriori bottleneck:** Even though the database is small, Apriori still performs level-wise candidate counting over many levels (up to size 20 in the logs). This makes the "checking subsets" phase the dominant cost.
- **FP-Growth efficiency:** FP-Growth avoids explicit candidate generation. In these runs, most of its time is spent in the final "writing" phase, which includes mining/enumeration and output emission of the (very large) frequent-itemset set.
- **Extreme support collapse:** At 90% support, almost no items meet the threshold in the synthetic dataset, causing both algorithms to finish extremely quickly

3.5 Key Insights on Item Distribution Impact

1. **Correlation density:** Dense, uniform correlations (synthetic) allow Apriori to maintain constant runtime since candidate generation doesn't increase with support. In contrast, WebDocs' complex distribution causes significant runtime variation.
2. **Itemset complexity:** WebDocs with 262 items generates far more frequent itemsets (217,767 at 10%) compared to the synthetic dataset's simpler pattern space. This amplifies Apriori's candidate generation overhead.
3. **Database compression:** Smaller, more structured datasets (synthetic) are more amenable to FP-tree compression, contributing to consistently high FP-Growth performance.
4. **Scalability implications:** For large, complex datasets like WebDocs, FP-Growth's advantage increases significantly, demonstrating better scalability for real-world applications.

4 Q1 Conclusions

4.1 Algorithm Performance Summary

1. **FP-Growth superiority:** FP-Growth consistently outperforms Apriori, with the most dramatic speedups (5-8×) observed at low to moderate support thresholds.

2. **Data-dependent performance:** The performance gap depends heavily on dataset characteristics:

   - Complex real-world data (WebDocs): Convergence at high support
   - Structured synthetic data: Stable speedup across support range

3. **Candidate generation overhead:** Apriori's main bottleneck is the exponential candidate generation at low support. FP-Growth's pattern growth approach avoids this entirely.

4. **Database scan efficiency:** FP-Growth's limited database scans provide significant advantages, especially as the database grows.

4.2 Practical Implications

- **Recommendation:** FP-Growth should be preferred for frequent itemset mining on realistic datasets, particularly when low support thresholds are required

- **Memory considerations:** FP-Growth trades time for memory (FP-tree storage), but the space complexity remains reasonable for most datasets

- **Implementation maturity:** Both algorithms are well-studied and implemented, but FP-Growth represents a modern, more efficient approach suitable for contemporary applications

5 Runtime Plots (Synthetic Dataset)

Figure 2 shows the runtime curve produced by re-running the Task 1 pipeline on the generated dataset.

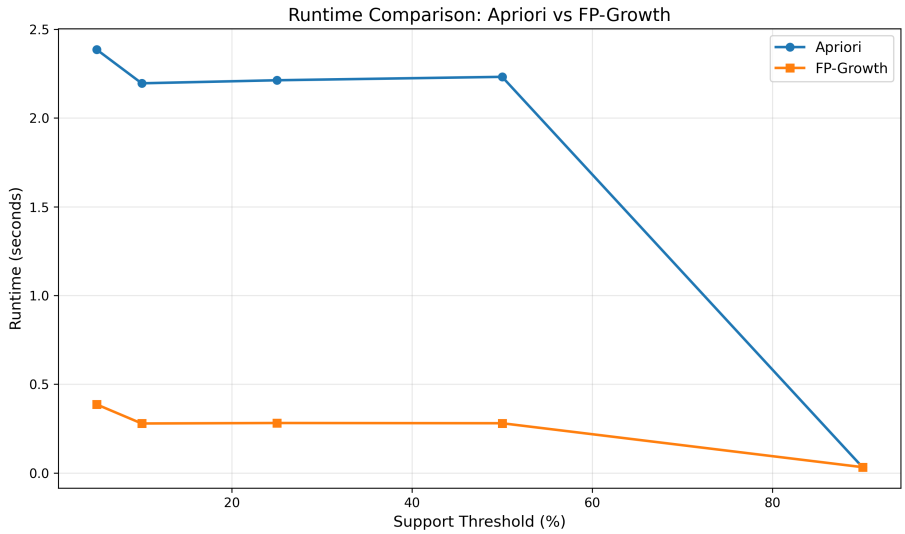

Figure 2: Runtime comparison on the synthetic dataset (`generated_transactions.dat`). X-axis: support threshold (%). Y-axis: runtime (seconds).

6 Reproducibility

- **Task 1:** `cd A1/q1` then run `bash q1_1.sh <apriori_exe> <fpgrowth_exe> <dataset> <outdir>`.
- **Task 2 dataset:** `cd A1/q1` then run `bash q1_2.sh <universal_itemset> <num_transactions>` to create `generated_transactions.dat`.
- **Plotting:** The pipeline calls a Python plotting script that reads `results.txt` and produces `plot.png` with axes labeled in % (support) and seconds (runtime).

7 References

- Apriori implementation and documentation (Christian Borgelt): `https://borgelt.net/src/apriori.zip`, `https://borgelt.net/doc/apriori/apriori.html`
- FP-Growth implementation and documentation (Christian Borgelt): `https://borgelt.net/src/fpgrowth.zip`, `https://borgelt.net/doc/fpgrowth/fpgrowth.html`

- WebDocs dataset (FIMI repository): `http://fimi.uantwerpen.be/data/webdocs.dat.gz`, description `http://fimi.uantwerpen.be/data/webdocs.pdf`